



NestJS Backend Assessment — Project Collaboration API



Assessment Overview

Duration

- Estimated Time: ~6 Hours

Objective

Build a scalable backend API using:

- NestJS
- MongoDB (Mongoose)
- JWT Authentication
- Role-Based Authorization

The assessment focuses on:

- NestJS architecture understanding
 - Clean backend structure
 - Validation & security
 - Real-world API design
 - Production-oriented practices
-



Use Case

Build a simple **Project Collaboration System** where users can create projects and manage issues/tasks within those projects.

The system should support:

- Authentication
- Role-based access
- Project management
- Issue tracking
- Reporting APIs

User Roles

1. Admin

Can:

- View all projects
- View all issues
- Manage users/projects

2. User

Can:

- Create projects
- Manage only their own projects
- Create/update issues inside accessible projects

Functional Requirements

Authentication & Authorization

Implement:

- JWT Authentication
- Role-based authorization using Guards

Requirements:

- Register
 - Login
 - Protected routes
 - Role-based access control
-

2 Project Management

Project Fields

- title (required)
- description
- owner
- members[]
- timestamps

APIs

Protected

- Create project
- Update own project
- Delete own project
- Get project details
- List projects

3 Issue Management

Each issue belongs to a project.

Issue Fields

- title
- description
- status
 - todo
 - in-progress
 - completed
- priority
 - low
 - medium
 - high
- assignedTo

- projectId
 - timestamps
-

APIs

- Create issue
 - Update issue
 - Delete issue
 - Get issue details
 - List issues with:
 - filtering
 - sorting
 - pagination
-

Reporting APIs

Create APIs for:

- Total issues by status
- Project-wise issue count
- Top users with most assigned issues

Use MongoDB Aggregation Pipeline where appropriate.

Technical Requirements

NestJS Concepts (MANDATORY)

The project should properly utilize:

- Modules
- Controllers
- Services
- Dependency Injection
- DTOs

- Validation Pipes
 - Guards
 - Interceptors
 - Exception Filters
 - ConfigModule
 - Environment variables
-

Database

Use:

- MongoDB + Mongoose

Expected:

- Proper schema design
 - Relations/references where needed
 - Validation
 - Timestamps
-

Validation

Use:

- class-validator
- DTO validation

Validation should return meaningful errors.

API Documentation

Implement:

- Swagger/OpenAPI documentation
-

Error Handling

-
- Centralized exception handling preferred
 - Proper HTTP status codes
 - Consistent response structure
-

Important Expectations

You are expected to think about:

- Folder structure
- Reusability
- Access control
- Edge cases
- Validation
- Clean architecture

Not every implementation detail is explicitly defined intentionally.

Edge Cases to Consider

At minimum:

- Invalid JWT
 - Unauthorized access
 - Accessing others' projects
 - Invalid Mongo IDs
 - Duplicate project names (optional)
 - Invalid pagination params
 - Missing environment variables
-

Non-Functional Requirements

Code Quality

- Proper naming conventions

• Proper naming conventions

- Clean folder structure
 - Separation of concerns
 - Reusable utilities where appropriate
-

Git Workflow

- Use Git properly
 - Meaningful commits
 - Feature-based branches preferred
-

Documentation

Provide:

- README.md
 - Setup instructions
 - Environment variable details
 - API overview
 - Assumptions/limitations
-

Tooling

Use:

- ESLint
 - Prettier
-



Bonus (Optional)

- Refresh token mechanism
- Custom decorators
- Global response interceptor
- Logger implementation
- Soft delete

- Unit tests



Evaluation Criteria

Area	Weight
Functionality	30%
NestJS Architecture	25%
Validation & Security	15%
Database Design	10%
Code Quality	10%
Edge Cases & Error Handling	10%



Expected Deliverables

1. GitHub Repository
2. NestJS Application
3. MongoDB Integration
4. Swagger Documentation
5. README.md
6. Postman Collection
7. Final Pull Request



Final Note

Focus on:

- Understanding
- Structure
- Clean implementation
- Backend engineering mindset

Perfection is not expected